

Python

Violet

2026-04-25

Chapter 0

目 录

I	Python 核心基础	
1.	Python 入门与环境搭建	6
2.	基础语法	7
3.	数据结构基础	8
3.1.	列表 (List)	8
3.2.	元组 (Tuple)	10
3.3.	字典 (Dict)	12
3.4.	集合 (Set)	14
3.5.	字符串高级操作	16
4.	函数与模块化	20
5.	面向对象编程 (OOP)	21
6.	异常处理	22
II	Python 高级特性与函数式编程	
7.	迭代器与生成器	24
7.1.	迭代协议	24
7.2.	生成器函数	26
7.3.	生成器表达式	28
7.4.	itertools 模块	29
8.	装饰器	33
9.	描述符与元类	34
10.	函数式编程	35
III	标准库与常用工具	
11.	文件系统与路径操作	37
12.	日期时间与时间处理	38
13.	正则表达式	39
14.	JSON 与数据序列化	40

15.	并发编程基础 ·····	41
16.	异步编程 (asyncio) ·····	42
17.	网络编程 ·····	43

IV

Python 工程化与最佳实践

18.	虚拟环境与依赖管理 ·····	45
19.	代码质量与规范 ·····	46
20.	测试与调试 ·····	47
21.	打包与发布 ·····	48

V

Python 底层原理与性能优化

22.	Python 对象模型 ·····	50
23.	内存管理与垃圾回收 ·····	51
24.	GIL 与并发模型 ·····	52
25.	性能分析与优化 ·····	53
26.	CPython 解释器架构 ·····	54



Python 核心基础

1. Python 入门与环境搭建 ·····	6
2. 基础语法 ·····	7
3. 数据结构基础 ·····	8
4. 函数与模块化 ·····	20
5. 面向对象编程 (OOP) ·····	21
6. 异常处理 ·····	22

Chapter 1

Python 入门与环境搭建

Chapter 2

基础语法

Chapter 3

数据结构基础

Python 提供了丰富的内置数据结构，包括列表、元组、字典和集合。掌握这些数据结构是 Python 编程的基础。

1 列表 (List)

列表是 Python 中最常用的可变序列类型。

1.1 创建与初始化

```
1 # 基本创建
2 fruits = ['apple', 'banana', 'cherry']
3 numbers = [1, 2, 3, 4, 5]
4 mixed = [1, 'hello', 3.14, True]
5
6 # 空列表
7 empty = []
8 empty = list()
9
10 # 列表推导式
11 squares = [x**2 for x in range(10)]
12 evens = [x for x in range(20) if x % 2 == 0]
13
14 # 嵌套列表
15 matrix = [
16     [1, 2, 3],
17     [4, 5, 6],
18     [7, 8, 9]
19 ]
```

TIP

列表推导式比传统循环更简洁、更高效，是 Python 的标志性特性。

1.2 切片操作

```
1 numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
2
3 # 基本切片
4 print(numbers[2:5])    # [2, 3, 4]
5 print(numbers[:3])     # [0, 1, 2]
```



```
6 print(numbers[7:])      # [7, 8, 9]
7
8 # 步长
9 print(numbers[::2])      # [0, 2, 4, 6, 8]
10 print(numbers[1::2])    # [1, 3, 5, 7, 9]
11
12 # 负数索引
13 print(numbers[-1])      # 9
14 print(numbers[-3:])     # [7, 8, 9]
15
16 # 反转列表
17 print(numbers[::-1])    # [9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
18
19 # 切片赋值
20 numbers[2:5] = [20, 30, 40]
21 print(numbers)          # [0, 1, 20, 30, 40, 5, 6, 7, 8, 9]
```

NOTE

切片返回新列表，不会修改原列表。切片赋值可以修改原列表。

1.3 常用方法

```
1 fruits = ['apple', 'banana']
2
3 # 添加元素
4 fruits.append('cherry')      # 末尾添加
5 fruits.insert(0, 'orange')   # 指定位置插入
6 fruits.extend(['grape', 'melon']) # 扩展列表
7
8 # 删除元素
9 fruits.remove('banana')      # 删除指定值
10 popped = fruits.pop()        # 删除并返回最后一个
11 popped = fruits.pop(0)       # 删除并返回指定位置
12 del fruits[1]                # 删除指定索引
13
14 # 查找
15 index = fruits.index('apple') # 查找索引
16 count = fruits.count('apple') # 统计出现次数
17
18 # 排序
19 numbers = [3, 1, 4, 1, 5, 9, 2, 6]
20 numbers.sort()                # 原地排序
21 numbers.sort(reverse=True)    # 降序
22 sorted_numbers = sorted(numbers) # 返回新列表
23
24 # 其他
25 fruits.reverse()              # 反转列表
26 fruits.clear()                # 清空列表
27 copied = fruits.copy()        # 浅拷贝
```

 Python

CAUTION

`sort()` 是原地排序，返回 `None`；`sorted()` 返回新列表，不修改原列表。

1.4 列表推导式进阶

```
1 # 基本推导式
2 squares = [x**2 for x in range(10)]
3
4 # 带条件
5 evens = [x for x in range(20) if x % 2 == 0]
6
7 # 嵌套推导式
8 pairs = [(x, y) for x in range(3) for y in range(3)]
9
10 # 条件表达式
11 labels = ['even' if x % 2 == 0 else 'odd' for x in range(10)]
12
13 # 处理嵌套列表
14 matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
15 flattened = [num for row in matrix for num in row]
16 # [1, 2, 3, 4, 5, 6, 7, 8, 9]
17
18 # 实际应用：过滤和转换
19 words = ['hello', 'world', 'python', 'programming']
20 upper_words = [w.upper() for w in words if len(w) > 5]
21 # ['PYTHON', 'PROGRAMMING']
```

 Python**TIP**

复杂的推导式会降低可读性，此时应使用传统循环。

2 元组 (Tuple)

元组是不可变的序列类型，适合存储固定数据。

2.1 创建与特性

```
1 # 基本创建
2 point = (3, 4)
3 color = ('red', 'green', 'blue')
4 single = (42,) # 单元素元组需要逗号
5 empty = ()
6
7 # 不使用括号
8 point = 3, 4
9
10 # 不可变性
11 t = (1, 2, 3)
12 # t[0] = 10 # TypeError: 不支持赋值
13
```

 Python

```
14 # 但可以包含可变对象
15 t = ([1, 2], [3, 4])
16 t[0].append(3) # 可以修改列表内容
17 print(t) # ([1, 2, 3], [3, 4])
```

NOTE

元组的不可变性使其可以作为字典的键，而列表不行。

2.2 解包赋值

```
1 # 基本解包
2 point = (3, 4)
3 x, y = point
4 print(x, y) # 3 4
5
6 # 交换变量
7 a, b = 1, 2
8 a, b = b, a # Pythonic 的交换方式
9
10 # 星号解包
11 first, *middle, last = [1, 2, 3, 4, 5]
12 print(first) # 1
13 print(middle) # [2, 3, 4]
14 print(last) # 5
15
16 # 函数返回多值
17 def get_name_age():
18     return 'Alice', 25
19
20 name, age = get_name_age()
21
22 # 嵌套解包
23 nested = (1, (2, 3), 4)
24 a, (b, c), d = nested
```

 Python**TIP**

解包赋值让代码更简洁，是 Python 的重要特性。

2.3 命名元组

```
1 from collections import namedtuple
2
3 # 定义命名元组
4 Point = namedtuple('Point', ['x', 'y'])
5 Color = namedtuple('Color', ['red', 'green', 'blue'])
6
7 # 创建实例
8 p = Point(3, 4)
9 c = Color(255, 128, 0)
10
```

 Python

```
11 # 访问字段
12 print(p.x, p.y)      # 3 4
13 print(c.red)        # 255
14
15 # 仍然支持索引
16 print(p[0], p[1])    # 3 4
17
18 # 转换为字典
19 print(p._asdict())   # {'x': 3, 'y': 4}
20
21 # 替换字段（返回新元组）
22 p2 = p._replace(x=10)
23 print(p2)           # Point(x=10, y=4)
24
25 # 从字典创建
26 data = {'x': 5, 'y': 10}
27 p3 = Point(**data)
```

NOTE

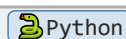
命名元组比普通元组更具可读性，比字典更轻量，适合表示简单记录。

3 字典 (Dict)

字典是键值对集合，提供 $O(1)$ 的查找性能。

3.1 创建与基本操作

```
1 # 基本创建
2 student = {
3     'name': 'Alice',
4     'age': 20,
5     'major': 'Computer Science'
6 }
7
8 # 空字典
9 empty = {}
10 empty = dict()
11
12 # 从键值对创建
13 student = dict(name='Alice', age=20, major='CS')
14
15 # 从列表创建
16 pairs = [('name', 'Alice'), ('age', 20)]
17 student = dict(pairs)
18
19 # 访问值
20 print(student['name'])    # Alice
21 print(student.get('age')) # 20
22 print(student.get('grade', 'N/A')) # N/A (默认值)
```



```
23
24 # 修改和添加
25 student['age'] = 21
26 student['grade'] = 'A'
27
28 # 删除
29 del student['grade']
30 age = student.pop('age')
31 last_item = student.popitem() # 删除最后一项
```

CAUTION

访问不存在的键会抛出 `KeyError`，建议使用 `get()` 方法提供默认值。

3.2 字典推导式

```
1 # 基本推导式
2 squares = {x: x**2 for x in range(5)}
3 # {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
4
5 # 带条件
6 even_squares = {x: x**2 for x in range(10) if x % 2 == 0}
7
8 # 键值转换
9 original = {'a': 1, 'b': 2, 'c': 3}
10 reversed_dict = {v: k for k, v in original.items()}
11 # {1: 'a', 2: 'b', 3: 'c'}
12
13 # 实际应用：统计词频
14 words = ['apple', 'banana', 'apple', 'cherry', 'banana', 'apple']
15 word_count = {word: words.count(word) for word in set(words)}
16 # {'apple': 3, 'banana': 2, 'cherry': 1}
```

 Python

3.3 defaultdict 和 OrderedDict

```
1 from collections import defaultdict, OrderedDict
2
3 # defaultdict: 自动初始化缺失键
4 word_count = defaultdict(int)
5 for word in ['apple', 'banana', 'apple']:
6     word_count[word] += 1
7 print(word_count) # defaultdict(<class 'int'>, {'apple': 2, 'banana': 1})
8
9 # 其他类型的 defaultdict
10 list_dict = defaultdict(list)
11 list_dict['fruits'].append('apple')
12 list_dict['fruits'].append('banana')
13
14 set_dict = defaultdict(set)
15 set_dict['tags'].add('python')
16 set_dict['tags'].add('programming')
```

 Python

```
17
18 # OrderedDict: 保持插入顺序 (Python 3.7+ 普通字典也保持顺序)
19 od = OrderedDict()
20 od['first'] = 1
21 od['second'] = 2
22 od['third'] = 3
23
24 # 移动到末尾
25 od.move_to_end('first')
26
27 # 弹出第一项
28 od.popitem(last=False)
```

TIP

Python 3.7+ 中，普通字典已经保持插入顺序，OrderedDict 主要用于需要顺序敏感操作的场景。

3.4 字典常用方法

```
1 student = {'name': 'Alice', 'age': 20, 'major': 'CS'}
2
3 # 遍历
4 for key in student:
5     print(key, student[key])
6
7 for key, value in student.items():
8     print(f'{key}: {value}')
9
10 # 合并字典 (Python 3.9+)
11 dict1 = {'a': 1, 'b': 2}
12 dict2 = {'c': 3, 'd': 4}
13 merged = dict1 | dict2
14
15 # 旧版本合并
16 merged = {**dict1, **dict2}
17 merged = dict1.copy()
18 merged.update(dict2)
19
20 # 其他方法
21 keys = student.keys()
22 values = student.values()
23 items = student.items()
24
25 exists = 'name' in student # True
26 size = len(student)        # 3
```

4 集合 (Set)

集合是无序的不重复元素集合，支持数学集合运算。

4.1 创建与基本操作

```
1 # 基本创建
2 fruits = {'apple', 'banana', 'cherry'}
3 numbers = set([1, 2, 3, 4, 5])
4 empty = set() # 注意: {} 是空字典
5
6 # 添加和删除
7 fruits.add('orange')
8 fruits.remove('banana') # 不存在会报错
9 fruits.discard('grape') # 不存在不报错
10 popped = fruits.pop() # 随机删除一个
11
12 # 集合推导式
13 squares = {x**2 for x in range(10)}
14 unique_lengths = {len(word) for word in ['hello', 'world', 'python']}
```

NOTE

集合常用于去重和 membership test (成员测试), 后者时间复杂度为 $O(1)$ 。

4.2 集合运算

```
1 A = {1, 2, 3, 4, 5}
2 B = {4, 5, 6, 7, 8}
3
4 # 并集
5 print(A | B) # {1, 2, 3, 4, 5, 6, 7, 8}
6 print(A.union(B))
7
8 # 交集
9 print(A & B) # {4, 5}
10 print(A.intersection(B))
11
12 # 差集
13 print(A - B) # {1, 2, 3}
14 print(A.difference(B))
15
16 # 对称差集
17 print(A ^ B) # {1, 2, 3, 6, 7, 8}
18 print(A.symmetric_difference(B))
19
20 # 子集和超集
21 C = {1, 2, 3}
22 print(C < A) # True (C 是 A 的子集)
23 print(A > C) # True (A 是 C 的超集)
24 print(C <= A) # True (C 是 A 的子集或相等)
25
26 # 不相交
27 D = {6, 7, 8}
28 print(A.isdisjoint(D)) # True
```

TIP

集合运算在处理数据去重、查找共同元素等场景非常有用。

4.3 frozenset

```
1 # frozenset 是不可变集合
2 frozen = frozenset([1, 2, 3, 4, 5])
3
4 # 可以作为字典的键
5 mapping = {frozenset([1, 2]): 'pair', frozenset([3, 4, 5]): 'triple'}
6
7 # 可以作为集合的元素
8 outer = {frozenset([1, 2]), frozenset([3, 4])}
9
10 # 不可修改
11 # frozen.add(6) # AttributeError
```

NOTE

frozenset 的不可变性使其可以哈希，因此可以作为字典的键或其他集合的元素。

5 字符串高级操作

5.1 split 和 join

```
1 # split: 分割字符串
2 text = "apple,banana,cherry"
3 fruits = text.split(',')
4 # ['apple', 'banana', 'cherry']
5
6 # 限制分割次数
7 path = "/usr/local/bin/python"
8 parts = path.split('/', 2)
9 # ['', 'usr', 'local/bin/python']
10
11 # rsplit: 从右边分割
12 filename = "archive.tar.gz"
13 name, ext = filename.rsplit('.', 1)
14 # name='archive.tar', ext='gz'
15
16 # splitlines: 按行分割
17 multiline = "line1\nline2\nline3"
18 lines = multiline.splitlines()
19 # ['line1', 'line2', 'line3']
20
21 # join: 连接字符串
22 fruits = ['apple', 'banana', 'cherry']
23 text = ', '.join(fruits)
24 # 'apple, banana, cherry'
25
```



```
26 # 实际应用: CSV 处理
27 csv_line = ','.join(['Alice', '25', 'Engineer'])
28 fields = csv_line.split(',')
```

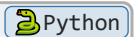
5.2 正则表达式

```
1  import re
2
3  # 基本匹配
4  pattern = r'\d{3}-\d{4}'
5  text = 'Phone: 123-4567'
6  match = re.search(pattern, text)
7  if match:
8      print(match.group()) # 123-4567
9
10 # 查找所有匹配
11 emails = re.findall(r'[\w.]+@[\w.]+', 'Contact: alice@example.com, bob@test.org')
12 # ['alice@example.com', 'bob@test.org']
13
14 # 替换
15 phone = re.sub(r'\d{3}-\d{4}', '***-****', 'Call 123-4567 or 890-1234')
16 # 'Call ***-**** or ***-****'
17
18 # 分组
19 pattern = r'(\d{4})-(\d{2})-(\d{2})'
20 text = 'Date: 2024-01-15'
21 match = re.search(pattern, text)
22 if match:
23     year, month, day = match.groups()
24     print(f'{year}年{month}月{day}日')
25
26 # 编译正则 (提高性能)
27 pattern = re.compile(r'\d+')
28 numbers = pattern.findall('There are 123 apples and 456 oranges')
29 # ['123', '456']
30
31 # 常用模式
32 patterns = {
33     'email': r'[\w.-]+@[\w.-]+\.\w+',
34     'phone': r'\d{3}-\d{4}|\d{11}',
35     'url': r'https?://[\w./-]+',
36     'ip': r'\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}',
37     'date': r'\d{4}-\d{2}-\d{2}',
38 }
```

CAUTION

使用原始字符串 (r'') 定义正则表达式, 避免转义字符问题。

5.3 文本处理技巧



```
1 # 大小写转换
2 text = "Hello World"
3 print(text.lower())      # hello world
4 print(text.upper())      # HELLO WORLD
5 print(text.title())      # Hello World
6 print(text.capitalize()) # Hello world
7 print(text.swapcase())   # hELLO wORLD
8
9 # 去除空白
10 text = "  hello  \n"
11 print(text.strip())      # hello
12 print(text.lstrip())     # hello  \n
13 print(text.rstrip())     #   hello
14
15 # 填充和对齐
16 text = "Python"
17 print(text.center(20, '-')) # -----Python-----
18 print(text.ljust(20, '-'))  # Python-----
19 print(text.rjust(20, '-'))  # -----Python
20 print(text.zfill(10))       # 0000Python
21
22 # 查找和替换
23 text = "Hello World"
24 print(text.find('World'))   # 6 (找不到返回-1)
25 print(text.index('World'))  # 6 (找不到抛出异常)
26 print(text.replace('World', 'Python')) # Hello Python
27 print(text.replace('l', 'L', 1))      # HeLlo World
28
29 # 判断方法
30 text = "Hello123"
31 print(text.isalpha())      # False
32 print(text.isdigit())      # False
33 print(text.isalnum())      # True
34 print(text.startswith('Hello')) # True
35 print(text.endswith('123'))    # True
36
37 # 格式化字符串
38 name = 'Alice'
39 age = 25
40 # f-string (推荐)
41 print(f'{name} is {age} years old')
42 # format 方法
43 print('{} is {} years old'.format(name, age))
44 # % 格式化 (旧式)
45 print('%s is %d years old' % (name, age))
```

TIP

f-string (Python 3.6+) 是最快、最可读的字符串格式化方式。

5.4 实战：文本处理示例

```
1  import re
2  from collections import Counter
3
4  # 词频统计
5  def word_frequency(text):
6      # 转换为小写并提取单词
7      words = re.findall(r'\b[a-zA-Z]+\b', text.lower())
8      return Counter(words)
9
10 text = "Python is great. Python is easy. Python is powerful."
11 freq = word_frequency(text)
12 print(freq.most_common(3))
13 # [('python', 3), ('is', 3), ('great', 1)]
14
15 # 验证邮箱格式
16 def validate_email(email):
17     pattern = r'^[\w.-]+@[ \w.-]+\.\w+$'
18     return bool(re.match(pattern, email))
19
20 print(validate_email('alice@example.com')) # True
21 print(validate_email('invalid@email'))    # False
22
23 # 提取 URL
24 def extract_urls(text):
25     pattern = r'https?://[\w.-]+'
26     return re.findall(pattern, text)
27
28 text = "Visit https://python.org or http://example.com"
29 urls = extract_urls(text)
30 print(urls) # ['https://python.org', 'http://example.com']
```

本章完

Chapter 4

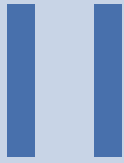
函数与模块化

Chapter 5

面向对象编程（OOP）

Chapter 6

异常处理



Python 高级特性与函数式编程

7. 迭代器与生成器	24
8. 装饰器	33
9. 描述符与元类	34
10. 函数式编程	35

Chapter 7

迭代器与生成器

迭代器和生成器是 Python 实现惰性求值和高效内存使用的核心机制。掌握它们是编写 Pythonic 代码的关键。

1 迭代协议

1.1 可迭代对象 vs 迭代器

```
1 # 可迭代对象 (Iterable)
2 # 实现了 __iter__ 方法, 返回迭代器
3 my_list = [1, 2, 3]
4 my_dict = {'a': 1, 'b': 2}
5 my_string = "hello"
6
7 # 检查是否可迭代
8 from collections.abc import Iterable
9 print(isinstance(my_list, Iterable)) # True
10
11 # 迭代器 (Iterator)
12 # 实现了 __iter__ 和 __next__ 方法
13 iterator = iter(my_list)
14 print(next(iterator)) # 1
15 print(next(iterator)) # 2
16 print(next(iterator)) # 3
17 # print(next(iterator)) # StopIteration
```

NOTE

所有迭代器都是可迭代对象，但并非所有可迭代对象都是迭代器。

1.2 手动实现迭代器

```
1 class Countdown:
2     """倒计时迭代器"""
3
4     def __init__(self, start):
5         self.start = start
6
7     def __iter__(self):
8         return self
9
```



```
10 def __next__(self):
11     if self.start <= 0:
12         raise StopIteration
13     current = self.start
14     self.start -= 1
15     return current
16
17 # 使用
18 for num in Countdown(5):
19     print(num, end=' ') # 5 4 3 2 1
```

1.3 自定义可迭代对象

```
1 class Fibonacci:
2     """斐波那契数列可迭代对象"""
3
4     def __init__(self, max_count=10):
5         self.max_count = max_count
6
7     def __iter__(self):
8         return FibonacciIterator(self.max_count)
9
10 class FibonacciIterator:
11     """斐波那契迭代器"""
12
13     def __init__(self, max_count):
14         self.max_count = max_count
15         self.count = 0
16         self.a, self.b = 0, 1
17
18     def __iter__(self):
19         return self
20
21     def __next__(self):
22         if self.count >= self.max_count:
23             raise StopIteration
24
25         result = self.a
26         self.a, self.b = self.b, self.a + self.b
27         self.count += 1
28         return result
29
30 # 使用
31 for num in Fibonacci(10):
32     print(num, end=' ') # 0 1 1 2 3 5 8 13 21 34
```

 Python

TIP

将迭代逻辑分离到独立的迭代器类中，使代码更清晰、更可维护。

2 生成器函数

生成器是一种特殊的迭代器，使用 `yield` 关键字实现。

2.1 yield 基础

```
1 def count_down(start):
2     """倒计时生成器"""
3     while start > 0:
4         yield start
5         start -= 1
6
7 # 使用
8 gen = count_down(5)
9 print(next(gen)) # 5
10 print(next(gen)) # 4
11
12 # for 循环自动处理 StopIteration
13 for num in count_down(5):
14     print(num, end=' ') # 5 4 3 2 1
```

NOTE

生成器函数调用时不会立即执行，而是返回一个生成器对象。每次调用 `next()` 时执行到下一个 `yield`。

2.2 惰性求值

```
1 def read_large_file(file_path):
2     """逐行读取大文件，避免一次性加载到内存"""
3     with open(file_path, 'r') as f:
4         for line in f:
5             yield line.strip()
6
7 # 使用
8 for line in read_large_file('huge_file.txt'):
9     process(line) # 逐行处理，内存占用极小
10
11 # 对比：传统方式会一次性加载整个文件
12 with open('huge_file.txt', 'r') as f:
13     lines = f.readlines() # 可能占用大量内存
```

CAUTION

生成器的惰性求值意味着数据只在需要时才生成，适合处理大数据流。

2.3 无限序列

```
1 def infinite_counter(start=0):
2     """无限计数器"""
3     current = start
```

```

4     while True:
5         yield current
6         current += 1
7
8     # 使用（需要手动停止）
9     counter = infinite_counter(10)
10    for _ in range(5):
11        print(next(counter), end=' ') # 10 11 12 13 14
12
13    # 斐波那契无限序列
14    def fibonacci():
15        a, b = 0, 1
16        while True:
17            yield a
18            a, b = b, a + b
19
20    # 取前10个斐波那契数
21    fib = fibonacci()
22    for _ in range(10):
23        print(next(fib), end=' ') # 0 1 1 2 3 5 8 13 21 34

```

2.4 send() 和协程

```

1  def accumulator():
2      """累加器生成器"""
3      total = 0
4      while True:
5          value = yield total
6          if value is not None:
7              total += value
8
9      # 使用
10     acc = accumulator()
11     next(acc) # 启动生成器, 输出 0
12     print(acc.send(10)) # 发送 10, 输出 10
13     print(acc.send(20)) # 发送 20, 输出 30
14     print(acc.send(30)) # 发送 30, 输出 60
15
16     # 关闭生成器
17     acc.close()

```

 Python

TIP

`send()` 可以向生成器发送值，实现双向通信，这是协程的基础。

2.5 yield from

```

1  def chain(*iterables):
2      """链式迭代多个可迭代对象"""
3      for iterable in iterables:
4          yield from iterable

```

 Python

```
5
6 # 使用
7 result = list(chain([1, 2, 3], 'abc', [4, 5]))
8 print(result) # [1, 2, 3, 'a', 'b', 'c', 4, 5]
9
10 # 递归生成器
11 def flatten(nested):
12     """扁平化嵌套列表"""
13     for item in nested:
14         if isinstance(item, (list, tuple)):
15             yield from flatten(item)
16         else:
17             yield item
18
19 nested = [1, [2, [3, 4]], [5, [6, [7]]]]
20 print(list(flatten(nested))) # [1, 2, 3, 4, 5, 6, 7]
```

NOTE

`yield from` 简化了从子生成器中委托迭代的代码，是 Python 3.3+ 的特性。

3 生成器表达式

生成器表达式是列表推导式的惰性版本。

3.1 基本语法

```
1 # 列表推导式（立即计算，占用内存）
2 squares_list = [x**2 for x in range(1000000)]
3
4 # 生成器表达式（惰性计算，节省内存）
5 squares_gen = (x**2 for x in range(1000000))
6
7 # 逐个获取
8 print(next(squares_gen)) # 0
9 print(next(squares_gen)) # 1
10
11 # 求和（不需要创建中间列表）
12 total = sum(x**2 for x in range(1000000))
```

**CAUTION**

生成器只能遍历一次，遍历完后就 exhausted（耗尽）了。

3.2 与列表推导式的对比

```
1 import sys
2
3 # 内存占用对比
4 list_comp = [x**2 for x in range(10000)]
```



```
5 gen_exp = (x**2 for x in range(10000))
6
7 print(sys.getsizeof(list_comp)) # ~87624 bytes
8 print(sys.getsizeof(gen_exp)) # ~200 bytes
9
10 # 性能对比
11 import time
12
13 # 列表推导式
14 start = time.time()
15 sum([x**2 for x in range(1000000)])
16 print(f"List: {time.time() - start:.4f}s")
17
18 # 生成器表达式
19 start = time.time()
20 sum(x**2 for x in range(1000000))
21 print(f"Generator: {time.time() - start:.4f}s")
22
23 # 生成器通常更快，因为不需要创建中间列表
```

TIP

当只需要遍历一次或配合聚合函数（sum/min/max）使用时，优先选择生成器表达式。

3.3 实际应用

```
1 # 过滤大文件
2 with open('large_file.txt') as f:
3     # 只处理包含 'error' 的行
4     error_lines = (line for line in f if 'error' in line.lower())
5     for line in error_lines:
6         process_error(line)
7
8 # 管道式数据处理
9 data = range(1000)
10 even_squares = (x**2 for x in data if x % 2 == 0)
11 filtered = (x for x in even_squares if x > 100)
12 total = sum(filtered)
13
14 # 矩阵转置
15 matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
16 transposed = list(zip(*matrix))
17 # [(1, 4, 7), (2, 5, 8), (3, 6, 9)]
```

 Python

4 itertools 模块

`itertools` 提供了高效的迭代工具，是 Python 标准库中的宝藏。

4.1 无限迭代器

```
1 import itertools
2
3 # count: 无限计数
4 counter = itertools.count(10, 2) # 从10开始, 步长2
5 print([next(counter) for _ in range(5)]) # [10, 12, 14, 16, 18]
6
7 # cycle: 无限循环
8 colors = itertools.cycle(['red', 'green', 'blue'])
9 print([next(colors) for _ in range(7)])
10 # ['red', 'green', 'blue', 'red', 'green', 'blue', 'red']
11
12 # repeat: 重复元素
13 repeated = itertools.repeat('A', 5)
14 print(list(repeated)) # ['A', 'A', 'A', 'A', 'A']
```

NOTE

无限迭代器需要配合 `islice` 或其他限制方式使用, 否则会无限循环。

4.2 有限迭代器

```
1 import itertools
2
3 # chain: 链式连接
4 result = list(itertools.chain([1, 2], 'abc', [3, 4]))
5 # [1, 2, 'a', 'b', 'c', 3, 4]
6
7 # compress: 根据选择器过滤
8 data = [1, 2, 3, 4, 5]
9 selectors = [True, False, True, False, True]
10 print(list(itertools.compress(data, selectors))) # [1, 3, 5]
11
12 # dropwhile / takewhile
13 nums = [1, 3, 5, 7, 2, 4, 6]
14 print(list(itertools.dropwhile(lambda x: x < 5, nums))) # [5, 7, 2, 4, 6]
15 print(list(itertools.takewhile(lambda x: x < 5, nums))) # [1, 3]
16
17 # filterfalse: 反向过滤
18 print(list(itertools.filterfalse(lambda x: x % 2 == 0, range(10))))
19 # [1, 3, 5, 7, 9]
20
21 # islice: 切片
22 print(list(itertools.islice(range(100), 10, 20, 2))) # [10, 12, 14, 16, 18]
```

4.3 组合迭代器

```
1 import itertools
2
3 # product: 笛卡尔积
4 result = list(itertools.product('AB', '12'))
5 # [('A', '1'), ('A', '2'), ('B', '1'), ('B', '2')]
```

```

6
7 # permutations: 排列
8 perms = list(itertools.permutations('ABC', 2))
9 # [('A', 'B'), ('A', 'C'), ('B', 'A'), ('B', 'C'), ('C', 'A'), ('C', 'B')]
10
11 # combinations: 组合
12 combs = list(itertools.combinations('ABC', 2))
13 # [('A', 'B'), ('A', 'C'), ('B', 'C')]
14
15 # combinations_with_replacement: 可重复组合
16 combs_rep = list(itertools.combinations_with_replacement('ABC', 2))
17 # [('A', 'A'), ('A', 'B'), ('A', 'C'), ('B', 'B'), ('B', 'C'), ('C', 'C')]

```

TIP

组合迭代器在解决排列组合问题时非常有用，比手写循环更高效。

4.4 其他实用工具

```

1  import itertools
2
3  # groupby: 分组（需要先排序）
4  data = [
5      {'name': 'Alice', 'age': 25},
6      {'name': 'Bob', 'age': 30},
7      {'name': 'Charlie', 'age': 25},
8  ]
9
10 # 按年龄分组
11 data.sort(key=lambda x: x['age'])
12 for age, group in itertools.groupby(data, key=lambda x: x['age']):
13     print(f"Age {age}: {[p['name'] for p in group]}")
14 # Age 25: ['Alice', 'Charlie']
15 # Age 30: ['Bob']
16
17 # starmap: 解包参数映射
18 result = list(itertools.starmap(pow, [(2, 3), (3, 2), (4, 2)]))
19 # [8, 9, 16]
20
21 # tee: 复制迭代器
22 iter1, iter2 = itertools.tee(range(5), 2)
23 print(list(iter1)) # [0, 1, 2, 3, 4]
24 print(list(iter2)) # [0, 1, 2, 3, 4]
25
26 # accumulate: 累积计算
27 result = list(itertools.accumulate([1, 2, 3, 4, 5]))
28 # [1, 3, 6, 10, 15] # 累积和
29
30 import operator
31 result = list(itertools.accumulate([1, 2, 3, 4, 5], operator.mul))
32 # [1, 2, 6, 24, 120] # 累积乘积

```

 Python

4.5 实战示例

```
1  import itertools
2
3  # 示例1: 生成所有可能的密码组合
4  def generate_passwords(chars, length):
5      """生成指定长度的所有密码组合"""
6      return itertools.product(chars, repeat=length)
7
8  passwords = generate_passwords('0123456789', 4)
9  for pwd in itertools.islice(passwords, 10): # 只看前10个
10     print(''.join(pwd))
11 # 0000, 0001, 0002, ...
12
13 # 示例2: 滑动窗口
14 def sliding_window(iterable, n):
15     """创建滑动窗口"""
16     it = iter(iterable)
17     window = tuple(itertools.islice(it, n))
18     if len(window) == n:
19         yield window
20     for elem in it:
21         window = window[1:] + (elem,)
22         yield window
23
24 data = [1, 2, 3, 4, 5, 6]
25 for window in sliding_window(data, 3):
26     print(window)
27 # (1, 2, 3), (2, 3, 4), (3, 4, 5), (4, 5, 6)
28
29 # 示例3: 合并多个有序序列
30 def merge_sorted(*sorted_sequences):
31     """合并多个已排序的序列"""
32     return itertools.merge(*sorted_sequences)
33
34 merged = list(itertools.merge([1, 3, 5], [2, 4, 6], [0, 7, 8]))
35 print(merged) # [0, 1, 2, 3, 4, 5, 6, 7, 8]
```

本章完

Chapter 8

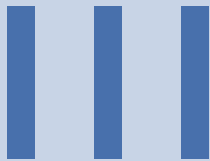
装饰器

Chapter 9

描述符与元类

Chapter 10

函数式编程



标准库与常用工具

11. 文件系统与路径操作 ·····	37
12. 日期时间与时间处理 ·····	38
13. 正则表达式 ·····	39
14. JSON 与数据序列化 ·····	40
15. 并发编程基础 ·····	41
16. 异步编程 (asyncio) ·····	42
17. 网络编程 ·····	43

Chapter 11

文件系统与路径操作

Chapter 12

日期时间与时间处理

Chapter 13

正则表达式

Chapter 14

JSON 与数据序列化

Chapter 15

并发编程基础

Chapter 16

异步编程（**asyncio**）

Chapter 17

网络编程

IV

Python 工程化与最佳实践

18. 虚拟环境与依赖管理 ·····	45
19. 代码质量与规范 ·····	46
20. 测试与调试 ·····	47
21. 打包与发布 ·····	48

Chapter 18

虚拟环境与依赖管理

Chapter 19

代码质量与规范

Chapter 20

测试与调试

Chapter 21

打包与发布



Python 底层原理与性能优化

22. Python 对象模型	50
23. 内存管理与垃圾回收	51
24. GIL 与并发模型	52
25. 性能分析与优化	53
26. CPython 解释器架构	54

Chapter 22

Python 对象模型

Chapter 23

内存管理与垃圾回收

Chapter 24

GIL 与并发模型

Chapter 25

性能分析与优化

Chapter 26

CPython 解释器架构
